

大模型文本蕴含能力的提升之道：基于海龟汤游戏的实证研究

王殿云 2022211733号 2022219107班

1. 摘要

本次作业中，笔者探究了提升大语言模型（LLM）文本蕴含能力的有效方法，以海龟汤游戏为实证研究载体，将其形式化为一种特殊的多轮文本蕴含任务。文本蕴含是自然语言处理中的基础语义关系，描述两个文本片段之间的推理逻辑联系。笔者提出并验证了两种提升大模型文本蕴含能力的方法：模型微调与多轮对话机制。在模型微调方面，笔者分别使用监督式微调（SFT）组相对策略优化（GRPO）两种技术对Qwen2.5-1.5B-Instruct模型进行了适应性调整。实验结果表明，相较于仅提升总体准确率23%的SFT方法，GRPO技术在处理矛盾和中立关系方面取得了质的飞跃，将矛盾关系识别准确率从15%提升至71%，中立关系从0%提升至63%。在多轮对话方面，笔者对比分析了Claude 3.7、GPT-4o、ChatGLM4以及前述本地微调模型在有无对话历史条件下的表现差异。实验显示对话历史对大多数模型的文本蕴含能力有显著增益，尤其是Claude 3.7和ChatGLM4分别获得了3.79%和8.02%的准确率提升，本地微调的Qwen模型更获得高达20.68%的性能提升。通过对错误案例的细致分析，笔者发现模型在社会关系、空间关系和抽象概念理解等方面仍存挑战。

2. 海龟汤游戏的概念与结构

海龟汤游戏是一种独特的思维谜题，其魅力在于展现了人类推理能力的精妙之处。在这个源自西方的"Lateral Thinking Puzzle"中，游戏构架围绕两个核心元素展开：表层信息（汤面）与深层真相（汤底）。汤面通常是一个令人困惑不解的故事片段，例如："一个男人走进一家餐厅，点了一碗海龟汤，喝完之后他就自杀了，请问为什么？"而汤底则是解开这一谜团的完整解释，包含了那些被刻意隐藏的关键信息，揭示了事件的真相。在游戏中，参与者分为两种角色：知晓完整信息的煲汤人（主持人）与只知道表面故事的喝汤人（玩家）。喝汤人通过提出一系列只能得到"是"、"否"或"与题目无关"三种回答的问题，逐步拼凑出隐藏在谜题背后的真相。这种限制性的问答模式既增加了游戏的挑战性，也形成了一种独特的推理结构，使得海龟汤成为研究语义理解和逻辑推理能力的绝佳载体。

2.1 核心概念：

汤面：一个令人难以理解的故事

- 例：一个男人走进一家餐厅，点了一碗海龟汤，喝完之后他就自杀了，请问为什么？

喝汤人（玩家）：

只知道汤面，提出一系列问题，对每个问题，等待煲汤人给出“是”，“否”，“与题目无关”三种回答之中的一个

- 例：这个男人以前吃的"海龟汤"是否真的是海龟做的？

汤底：汤面故事的真相

1. 例

他和妻子度蜜月时遭遇海难，流落荒岛，由于没有粮食，妻子被饿死，同伴用妻子的肉煮汤给他喝，骗他是海龟汤。后来他被路过的船只救走，今天，他喝到真正的海龟汤，才想起来当时吃下的是妻子的肉，悔恨之下举枪自尽。

煲汤人（主持人）：

提出一个令人难以理解的汤面，知道汤面和汤底，对喝汤人的问题给出“是”，“否”，“与题目无关”三种回答之中的一个

- 例：不是。

3. 从海龟汤到文本蕴涵任务

文本蕴涵关系是自然语言处理领域中的一个基础性语义关系，它描述了两个文本片段之间的推理逻辑联系。当一个文本作为前提（premise）时，若能从中合理推导出另一个作为假设（hypothesis）的文本，我们就称前提蕴涵假设，记为 $P \rightarrow H$ 。文本蕴涵任务的核心在于判断这种关系的存在性，通常分为三种可能的情况：蕴涵（entailment）、矛盾（contradiction）或无关（neutral）。海龟汤游戏与文本蕴涵任务之间存在着惊人的结构相似性，这使得我们能够构建一个自然的理论映射。在海龟汤中，煲汤人实际上是在执行一种特殊形式的文本蕴涵判断：将汤面和汤底的完整信息作为前提P，将玩家提出的每个问题视为假设H，然后判断二者之间的蕴涵关系，并基于此给出"是"、"否"或"与题目无关"的回答。这种映射不仅为我们提供了一个研究文本蕴涵能力的新颖视角，也为构建更具挑战性的评估任务提供了理论基础。本次作业将海龟汤游戏形式化为一个多轮文本蕴涵任务，其中系统需要根据完整解释（汤底）和先前的对话历史，判断新问题的答案，从而评估大型语言模型处理复杂语义推理的能力。

海龟汤煲汤人的工作其实与文本蕴涵任务高度重合：将汤面和汤底合并，作为P，将玩家的提问作为H，则煲汤人实际的工作是在对P与H的文本蕴涵关系做出回答。

例：

汤面+汤底	问题	煲汤人的回答
汤面：我和男友是校友，他比我大三届，马上就要毕业了。我总感觉和他心有灵犀，一见钟情，于是我们约好了等我到了法定结婚年龄马上结婚，快要过国庆了，	男友父母	是

男友对我说，不如假期我不要回去了，来我家见见我父母吧。我欣然同意。然而，男友的父母一看到我的脸，大惊失色，坚决反对我们结婚交往。这是为什么呢？ 汤底：这是刚刚改革开放，计划生育的年代。我是超生的孩子，为了保住工作，生下我之后父母把我送到乡下给远方亲戚抚养，并且改了姓。父母一直对亲戚进行着经济援助，得知我考上了这里的大学父母也很开心，而且也快要退休了。正在考虑是不是要联系到我让我回到这个家庭，谁知道我们亲兄妹竟然相爱了，人间悲剧。	是否认识女孩或女孩的家人？	
P	H	文本蕴含关系
汤面：我和男友是校友，他比我大三届，马上就要毕业了。我总感觉和他心有灵犀，一见钟情，于是我们约好了等我到了法定结婚年龄马上结婚，快要过国庆了，男友对我说，不如假期我不要回去了，来我家见见我父母吧。我欣然同意。然而，男友的父母一看到我的脸，大惊失色，坚决反对我们结婚交往。这是为什么呢？ 汤底：这是刚刚改革开放，计划生育的年代。我是超生的孩子，为了保住工作，生下我之后父母把我送到乡下给远方亲戚抚养，并且改了姓。父母一直对亲戚进行着经济援助，得知我考上了这里的大学父母也很开心，而且也快要退休了。正在考虑是不是要联系到我让我回到这个家庭，谁知道我们亲兄妹竟然相爱了，人间悲剧。	男友父母认识女孩或女孩的家人	P→H

本次作业将海龟汤游戏形式化为一个多轮文本蕴含任务。在这个任务中：

- 给定一个谜题 P ，包含表面描述（汤面） S 和完整解释（汤底） B 。
- 在第 t 轮对话中，玩家提出问题 q_t 。
- 系统需要根据完整解释 B 和先前的对话历史 $H_{t-1} = (q_1, a_1), (q_2, a_2), \dots, (q_{t-1}, a_{t-1})$ ，判断问题 q_t 的答案 $a_t \in \{\text{是}, \text{不是}, \text{这个问题与解谜无关}\}$ 。

笔者的目标是评估对话历史 H_{t-1} 对大型语言模型执行上述任务能力的影响。

4. 如何提升大语言模型的文本蕴涵能力

语义蕴涵任务本质上是一种复杂的推理（Reasoning）挑战，需要模型理解文本间的深层逻辑关联。目前学术界和产业界已经发展出多种提升大语言模型推理能力的方法论体系。Prompt工程作为最直接的干预手段，通过精心设计的提示模板引导模型进行结构化思考，有效激活了模型已有的潜在能力。而模型微调则采取更为积极的训练策略，通过高质量的任务专属数据调整模型参数，使其更好地适应特定领域的推理需求。除了这些静态优化手段外，多轮对话机制提供了一种动态增强模型理解的途径，允许模型在交互过程中积累上下文信息，逐步精炼其推理结果。更为先进的Agents System则引入了多智能体协作的思路，通过模拟认知分工与协作，突破单一模型的能力边界。这些方法并非相互排斥，而是形成了一个互补的技术谱系，共同推动大语言模型在语义蕴涵任务上的能力边界不断拓展。在实际应用中，这些方法的组合使用往往能够产生超越单一方法的性能提升，为解决复杂的语义理解问题提供了多维度的解决方案。

本作业中，笔者拟探究模型微调与多轮对话两种方式对大模型文本蕴含能力的提升，这两种方式的具体介绍如下。

4.1 模型微调

4.1.1 传统有监督微调：SFT

监督式微调(Supervised Fine-Tuning)是在预训练模型基础上，使用高质量人工标注数据进行额外训练，引导模型产生符合人类偏好的输出。

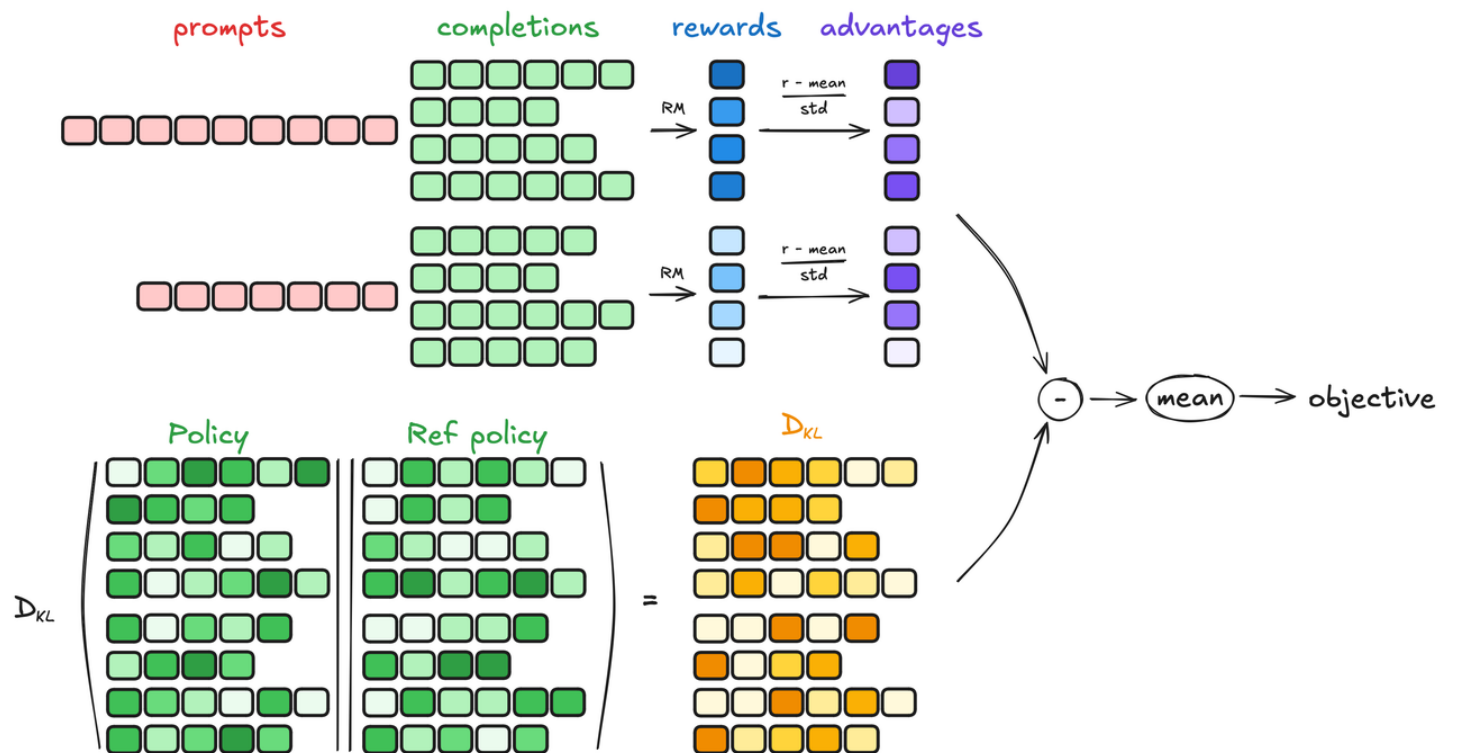
SFT的核心是最小化预测损失函数：

$$\mathcal{L}_{\text{SFT}} = - \sum_{(x, y) \in D} \sum_{i=1}^{|y|} \log P_{\theta}(y_i | x, y_{<i})$$

其中：

- (x, y) 是输入-输出对，来自标注数据集 D
- P_{θ} 是模型参数为 θ 的语言模型
- y_i 是目标输出序列中的第 i 个token
- $y_{<i}$ 表示目标输出序列中 i 之前的所有tokens

4.1.2 强化学习方法：GRPO



$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q, o_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right] - \beta D_{KL}[\pi_{\theta} || \pi] \right\}$$

GRPO是DeepSeek-R1系列模型得以成功的关键技术，是一种**在线学习**算法，通过使用训练过程中模型自身生成的数据进行迭代改进。其四个主要步骤是：

1. 采样

$$\mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

- **图中对应：**左侧的"prompts"(粉色方块)和"completions"(绿色方块)
- 粉色方块表示从分布P(Q)中采样的提示q
- 每个提示对应的多个绿色方块表示生成的G个完成内容

2. 奖励和优势值计算

$$\hat{A}_{i,t} = \frac{r_i - \mu(\mathbf{r})}{\sigma(\mathbf{r})}$$

- **图中对应：**中间的"rewards"(蓝色方块)到"advantages"(紫色方块)的转换
- 图中标注的"r-mean"和"std"正是公式中的 $\mu(\mathbf{r})$ 和 $\sigma(\mathbf{r})$
- 这个过程将原始奖励标准化为优势值

3. KL散度计算

$$\beta D_{KL}[\pi_{\theta} || \pi_{ref}]$$

- **图中对应：**底部的三个矩阵
- 左侧"Policy"(深绿色矩阵)代表当前策略 π_{θ}
- 中间"Ref policy"(浅绿色矩阵)代表参考策略 π_{ref}
- 右侧"D_KL"(黄橙色矩阵)表示它们之间的token级KL散度

4. 目标函数组合

$$...\hat{A}_{i,t} - \beta D_{KL}[\pi_{\theta} || \pi_{ref}]$$

- **图中对应：**右侧的减法运算(-)、平均值计算(mean)和最终目标(objective)
- 减法符号对应公式中的减号，将优势部分和KL散度部分结合
- "mean"对应公式中的平均计算： $\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|}$

4.2 多轮对话

多轮对话技术允许模型在连续交互中保持上下文信息，根据历史对话优化当前响应，已被证明能有效增强大模型的推理能力。

1. 信息累积效应

- 每轮对话都为模型提供新信息，丰富推理基础
- 实现了 $I(r_t; H) > 0$ （响应与历史的互信息为正）

2. 渐进式理解深化

- 用户提问 → 模型回答 → 用户澄清 → 模型调整
- 形成"理解-验证-修正"循环，逐步提高响应质量

3. 记忆和推理增强

- 通过记忆关键信息实现长程依赖建模

5. 数据集

5.1 The Stanford Natural Language Inference (SNLI) Dataset

5.1.1 数据集简介

斯坦福自然语言推理(SNLI)数据集是一个大规模标注数据集，专为自然语言推理(NLI)任务而设计。它由斯坦福大学研究人员于2015年发布，包含约57万对英语句子，每对句子被标注为三种推理关系之一：蕴含(entailment)、矛盾(contradiction)或中立(neutral)。

数据集的创建基于Flickr30k图像描述，通过众包方式收集。每对句子包含一个"前提句"(premise)和一个"假设句"(hypothesis)，模型需要判断前提与假设之间的逻辑关系。

5.1.2 数据集格式与示例

▼ 代码块

```
1  {
2      "annotator_labels": ["neutral"],
3      "captionID": "3416050480.jpg#4",
4      "gold_label": "neutral",
5      "pairID": "3416050480.jpg#4r1n",
6      "sentence1": "A person on a horse jumps over a broken down airplane.",
7      "sentence1_binary_parse": "( ( ( A person ) ( on ( a horse ) ) ) ( (
jumps ( over ( a ( broken ( down airplane ) ) ) ) ) . ) )",
      "sentence1_parse": "(ROOT (S (NP (NP (DT A) (NN person))) (PP (IN on) (NP (DT
a) (NN horse)))) (VP (VBZ jumps) (PP (IN over) (NP (DT a) (JJ broken) (JJ
down) (NN airplane)))) (. .)))",
8      "sentence2": "A person is training his horse for a competition.",
9      "sentence2_binary_parse": "( ( A person ) ( ( is ( ( training ( his horse
) ) ( for ( a competition ) ) ) ) . ) )",
10     "sentence2_parse": "(ROOT (S (NP (DT A) (NN person)) (VP (VBZ is) (VP
(VBG training) (NP (PRP$ his) (NN horse)) (PP (IN for) (NP (DT a) (NN
competition)))) (. .)))"
11 }
```

5.2 再来一口海龟汤：自主构建的海龟汤多轮对话推理数据集

5.2.1 数据集简介

"再来一口海龟汤：海龟汤多轮对话推理数据集"是**笔者自主构建的**，基于中文"海龟汤"(Lateral Thinking Puzzle)游戏构建的文本蕴含任务数据集。该数据集包含31个海龟汤故事，每个故事由表面信息（汤面）和完整解释（汤底）组成，并通过大型语言模型生成的思考链（Chain of Thought）模拟了一个解谜者通过多轮提问逐步揭示真相的推理过程。

5.2.2 数据集构造方法

5.2.2.1 原始数据

收集了31个经典海龟汤故事，每个故事包含标题、汤面和汤底

5.2.2.2 多轮对话生成过程

1. **使用模型**：Claude 3.7 Sonnet
2. **系统提示设计**：指导模型扮演海龟汤解谜者，从完全不知情的状态开始，通过多轮提问逐步接近真相
3. **结构化输出**：
 - 初始分析：对汤面的初步分析和可能的推理方向
 - 多轮问答（5-8轮）：每轮包含当前理解、提出问题、假设回答、分析回答、更新理解
 - 最终推理：综合分析、验证性问题、最终结论

5.2.3 数据集特点

1. **渐进式推理过程**：展示从有限信息到完整理解的认知演变
2. **结构化思考链**：每个推理步骤清晰标记，便于分析和学习
3. **多样化推理类型**：
 - 因果推理：分析事件原因和后果
 - 常识推理：利用现实世界知识进行判断
 - 意图推理：解读人物行为背后的动机
 - 时序推理：重建事件发生的顺序和背景
4. **文化和历史背景**：部分故事涉及特定历史或社会背景（如计划生育政策）
5. **情感和伦理元素**：包含情感冲突和伦理两难的复杂场景

5.2.4 数据集格式与示例

▼ 1代码块

```
2      "title": "见家长",
3      "surface": "我和男友是校友，他比我大三届，马上就要毕业了。我总感觉和他心有灵犀，一见钟情，于是我们约好了等我到了法定结婚年龄马上结婚，快要过国庆了，男友对我说，不如假期我不要回去了，来我家见见我父母吧。我欣然同意。然而，男友的父母一看到我的脸，大惊失色，坚决反对我们结婚交往。这是为什么呢？",
4      "bottom": "这是刚刚改革开放，计划生育的年代。我是超生的孩子，为了保住工作，生下我之后父母把我送到乡下给远方亲戚抚养，并且改了姓。父母一直对亲戚进行着经济援助，得知我考上了这里的大学父母也很开心，而且也快要退休了。正在考虑是不是要联系到我让我回到这个家庭，谁知道我们亲兄妹竟然相爱了，人间悲剧。",
5      "thought_chain": {
6          "initial_analysis": "这个故事涉及一对年轻恋人，女孩想与男友见家长，但男友父母见到女孩后强烈反对两人交往。这种反对似乎与女孩的外表有关（\ "一看到我的脸\ "），可能涉及到家庭背景、社会认同或其他隐藏因素。我需要探索男友父母反对的真正原因，可能的方向包括：身份冲突、外貌特征、家族关系或过去的连接。",
7          "rounds": [
8              {
9                  "current_understanding": "男友父母看到女孩的脸后坚决反对，可能与女孩的外貌或身份有关。",
10                 "question": "男友父母是否认识女孩或女孩的家人？",
11                 "answer": "是",
12                 "analysis": "这个回答非常关键，表明男友父母与女孩或女孩家庭之间存在某种联系。这可能解释他们为何一看到女孩的脸就反应强烈。可能是过去的恩怨、商业纠纷或其他关系。",
13                 "updated_understanding": "男友父母与女孩或女孩家人之间存在某种联系或认识关系，这可能是他们反对这段关系的根本原因。"
14             },
15             {
16                 "current_understanding": "男友父母与女孩或女孩家人之间存在某种联系。",
17                 "question": "故事是否发生在特殊的社会背景或历史时期？",
18                 "answer": "是",
19                 "analysis": "故事背景的特殊性可能是解谜的关键。根据提示，可能涉及某个特定的社会政策、历史事件或社会现象，这对故事发展有重要影响。",
20                 "updated_understanding": "故事发生在某个特殊的社会或历史背景下，这可能与男友父母反对的原因直接相关。需要进一步确定具体的时代背景。"
21             },
22             {
23                 "current_understanding": "故事发生在特殊社会/历史背景下，男友父母与女孩有某种联系。",
24                 "question": "故事是否发生在计划生育政策严格执行的年代？",
25                 "answer": "是",
26                 "analysis": "计划生育政策是中国特定历史时期的重要社会背景，可能与这个故事密切相关。在那个时期，超生或违反计划生育可能带来严重后果。",
27                 "updated_understanding": "故事发生在计划生育政策年代，这可能与女孩的身份或家庭背景有关。男友父母认出女孩可能涉及某种与计划生育相关的隐情。"
28             },
29             {
```

```

30         "current_understanding": "故事发生在计划生育时期，男友父母认识女孩，两者之
    间可能有隐藏关系。",
31         "question": "女孩是否可能是被送养的孩子？",
32         "answer": "是",
33         "analysis": "这是一个重要突破。在计划生育时期，一些超生的孩子被送给亲戚或他
    人抚养是存在的情况。女孩可能是被送养的孩子，而男友父母可能知道这一点。",
34         "updated_understanding": "女孩很可能是被送养的孩子，这与计划生育政策背景相
    符。男友父母认出她可能因为知道她的真实身份或家庭背景。"
35     },
36     {
37         "current_understanding": "女孩是被送养的孩子，故事发生在计划生育时期，男友
    父母认出了她。",
38         "question": "男友父母是否与女孩的亲生父母有关系？",
39         "answer": "是",
40         "analysis": "这个答案进一步缩小了可能性。男友父母与女孩亲生父母之间存在某种
    关系，这解释了为何他们会认出女孩并强烈反对这段恋情。",
41         "updated_understanding": "男友父母与女孩亲生父母有关系，可能是朋友、同事或
    亲戚。这一关系是他们认出女孩并反对恋情的原因。"
42     },
43     {
44         "current_understanding": "女孩是被送养的孩子，男友父母与女孩亲生父母有关
    系。",
45         "question": "女孩的亲生父母是否就是男友的父母？",
46         "answer": "是",
47         "analysis": "这个惊人的事实解释了所有疑点！女孩实际上是男友父母的亲生女儿，
    被送养出去，而两个年轻人不知道他们实际上是兄妹。这就解释了为什么男友父母一看到女孩就激烈
    反对这段关系。",
48         "updated_understanding": "女孩是男友的亲妹妹，两人不知情而坠入爱河。男友父
    母认出了自己曾经送养的女儿，所以坚决反对这段关系。"
49     },
50     {
51         "current_understanding": "女孩是男友父母的亲生女儿，被送养出去，现在与亲哥
    哥不知情地恋爱了。",
52         "question": "女孩被送养是否因为她是超生的孩子，父母为保住工作不得不送养？",
53         "answer": "是",
54         "analysis": "这完全符合计划生育时期的社会背景。父母为了保住工作，不得不将超
    生的孩子送养，现在这个悲剧性的局面出现了。",
55         "updated_understanding": "女孩是男友父母超生的孩子，为保住工作被送养到远方
    亲戚家，改姓后长大成人。她与亲哥哥相遇并相爱，却不知道彼此的真实关系。"
56     }
57 ],
58 "conversation_history": [

```

6. 强化学习与有监督学习

6.1 实验环境

本研究实验依托优云智算云计算平台开展，硬件配置采用4张NVIDIA GeForce RTX 4090 GPU并行加速器进行模型训练与微调。实验环境基于Ubuntu 22.04 LTS操作系统，深度学习框架选用PyTorch 2.0实现算法构建与计算加速。

6.2 实验设计

本研究采用自然语言推理任务中的标准基准数据集SNLI (Stanford Natural Language Inference)。SNLI数据集包含人工标注的句子对，每对包含一个前提(premise)和一个假设(hypothesis)，以及它们之间的逻辑关系标签：蕴含(entailment)、矛盾(contradiction)或中性(neutral)。笔者在snli-train上分别使用了SFT和GRPO方法微调了Qwen2.5-1.5B-Instruct模型。

具体微调方法如下：

6.2.1 SFT

监督式微调(Supervised Fine-Tuning, SFT)是大语言模型适应特定任务的基础方法。

6.2.1.1 数据构建与处理

对于本任务，笔者将每个训练样本构造为指令格式，包含任务描述、输入（前提和假设）以及期望输出（正确的推理关系），形成完整的指令-响应对。

通过 `prepare_dataset` 函数，笔者将原始SNLI样本转换为训练所需的格式，每个样本包含格式化的提示(`prompt`)和正确答案(`answer`)，同时按95:5的比例划分训练集和验证集，验证集最多包含500个样本

6.2.1.2 训练框架与主训练循环

监督式微调(SFT)阶段采用标准的语言模型训练方法，框架主要包括：

- 损失函数：**使用交叉熵损失(Cross Entropy Loss)作为优化目标，最小化模型预测分布与真实标签分布之间的差异。
- 优化器选择：**采用AdamW优化器，初始学习率设为 $5e-5$ ，并使用线性学习率调度器(Linear Scheduler)，带有预热步骤(warmup steps)。
- 训练策略：**
 - 批次大小：16（每个设备8，在2个GPU上并行）
 - 累积梯度步骤：4，有效扩大批量大小至64
 - 训练轮次：3，每轮次对整个训练集进行一次完整遍历
 - 最大序列长度：限制为512个令牌，平衡训练效率与上下文保留
- 正则化技术：**
 - 权重衰减：0.01，防止过拟合
 - 梯度裁剪：最大范数1.0，避免梯度爆炸

- 混合精度训练：使用torch.bfloat16格式，提升训练速度并减少内存占用

SFT阶段通过 `Trainer` 类实现，该类封装了模型训练的完整流程，包括批次准备、前向传播、损失计算、梯度更新和评估。每个训练步骤都会记录损失值，每500步进行一次验证评估，以监控训练进度和防止过拟合。由于SFT方法较为普遍，本处不过多赘述。

6.2.2 GRPO

采用组相对策略优化(Group Relative Policy Optimization, GRPO)算法进行强化学习微调。GRPO是一种强化学习方法，通过定义适当的奖励函数来引导模型生成期望的输出。

6.2.2.1 数据构建与处理

数据的主要处理流程类似于SFT。但为了引导模型学习自然语言推理任务，笔者在GRPO部分设计了结构化的提示模板。系统提示如下：

▼ SYSTEM PROMPT

```
1  请按照以下格式回答：
2  <reasoning>
3  分析前提和假设之间的关系。考虑以下几点：
4  如果基于前提，假设一定为真，关系为"entailment"
5  如果基于前提，假设一定为假，关系为"contradiction"
6  如果基于前提，无法确定假设的真假，关系为"neutral"
7  </reasoning>
8  <answer>
9  entailment or contradiction or neutral
10 </answer>
```

通过 `format_snli_prompt` 函数，笔者将每对句子转换为结构化提示：

1. 首先是系统提示，说明任务和回答格式
2. 然后是用户提示，包含标记清晰的前提和假设
3. 最后是请求模型判断关系类型的引导语

这种格式设计有两个目的：一方面引导模型进行推理，另一方面便于从模型输出中提取判断结果。

6.2.2.2 训练框架与主训练循环

6.2.2.2.1 奖励函数设计

GRPO的关键是设计合适的奖励函数。笔者为SNLI任务设计了两种奖励成分：

1. 准确性奖励(`snli_reward`):

- 当模型输出的推理判断与金标签一致时，给予1.0的奖励

- 当判断错误或无法提取判断时，给予0.0的奖励

2. 格式奖励(`format_reward`):

- 对于包含指定思考XML格式的回答，给予0.2的额外奖励

最终通过 `combined_reward` 函数将两种奖励合并，既鼓励模型生成准确的判断，也鼓励其生成简洁有效的回答。

6.2.2.2.2 GRPO核心组件

GRPO实现包含几个关键组件:

1. logits计算与概率优化:

- `selective_log_softmax` 计算特定词汇表中令牌的对数概率
- `compute_log_probs` 计算一批令牌的对数概率

2. 生成与展开:

- `generate_completions` 为每个提示生成多个候选回答
- `create_completion_mask` 创建完成令牌的掩码，处理EOS后的令牌
- `generate_rollout_data` 生成展开数据，包括完成和对数概率

3. 损失计算:

- `grpo_loss` 计算GRPO损失，包括策略比率截断和KL散度正则化
- 通过标准化优势值稳定训练

6.2.2.2.3 训练循环实现

训练框架采用了双层循环结构:

6.2.2.2.3.1 外循环

外循环控制GRPO的整体迭代过程，每次迭代都:

1. 创建当前模型的深拷贝作为参考模型(reference model)，并冻结其参数
2. 初始化新的AdamW优化器，学习率设为2e-6
3. 设置模型为训练模式

6.2.2.2.3.2 内循环

内循环执行具体的训练步骤，每个步骤:

1. 从训练数据中随机抽取batch_size(4)个样本
2. 生成rollout数据，包括为每个提示生成num_generations(8)个候选回答
3. 对每个批次进行多次($\mu=2$)策略更新:
 - 计算GRPO损失

- 清零梯度
- 反向传播
- 梯度裁剪(max_norm=0.1)
- 优化器步进
- 记录损失和奖励到Weights & Biases

为优化训练效率，本处还实现了多项内存优化措施：

- 使用 `optimize_model_memory` 函数减少内存占用
- 禁用缓存，启用梯度检查点(gradient checkpointing)
- 使用DataParallel实现多GPU并行训练

6.2.2.2.4 GRPO主要逻辑的具体实现

▼ 代码块

```

1  # 核心GRPO函数
2  def selective_log_softmax(logits, input_ids):
3      """计算特定词汇表中令牌的对数概率"""
4      log_probs = nn.functional.log_softmax(logits, dim=-1)
5      return log_probs.gather(dim=-1, index=input_ids.unsqueeze(-1)).squeeze(-1)
6
7  def compute_log_probs(model, input_ids, attention_mask, logits_to_keep):
8      """计算一批令牌的对数概率"""
9      logits = model(input_ids=input_ids,
10                     attention_mask=attention_mask).logits[:, :-1, :]
11      input_ids = input_ids[:, 1:] # 移除第一个令牌
12      input_ids = input_ids[:, -logits_to_keep:]
13      logits = logits[:, -logits_to_keep:, :]
14      return selective_log_softmax(logits, input_ids)
15
16  def create_completion_mask(completion_ids, eos_token_id):
17      """创建完成令牌的掩码，排除EOS令牌后的令牌"""
18      is_eos = completion_ids == eos_token_id
19      eos_idx = torch.full((is_eos.size(0),), is_eos.size(1), dtype=torch.long,
20                          device=completion_ids.device)
21      mask_exists = is_eos.any(dim=1)
22      eos_idx[mask_exists] = is_eos.int().argmax(dim=1)[mask_exists]
23      sequence_indices = torch.arange(is_eos.size(1),
24                                     device=completion_ids.device).expand(is_eos.size(0), -1)
25      return (sequence_indices <= eos_idx.unsqueeze(1)).int()
26
27  def generate_completions(model, tokenizer, prompts, num_generations=4,
28                          max_completion_length=32):
29      """为每个提示生成多个完成"""

```

```

26     device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
27     inputs = tokenizer(prompts, return_tensors="pt", padding=True,
padding_side="left")
28     prompt_ids = inputs["input_ids"].to(device)
29     prompt_mask = inputs["attention_mask"].to(device)
30
31     prompt_length = prompt_ids.size(1)
32     prompt_ids = prompt_ids.repeat_interleave(num_generations, dim=0)
33     prompt_mask = prompt_mask.repeat_interleave(num_generations, dim=0)
34
35     outputs = model.generate(
36         prompt_ids,
37         attention_mask=prompt_mask,
38         max_new_tokens=max_completion_length,
39         do_sample=True,
40         temperature=1.0,
41         pad_token_id=tokenizer.pad_token_id,
42         eos_token_id=tokenizer.eos_token_id,
43     )
44
45     completion_ids = outputs[:, prompt_length:]
46     completion_mask = create_completion_mask(completion_ids,
tokenizer.eos_token_id)
47
48     return prompt_ids, prompt_mask, completion_ids, completion_mask
49
50 def generate_rollout_data(model, ref_model, tokenizer, batch_samples,
num_generations, max_completion_length):
51     """生成GRPO展开数据, 包括完成和对数概率"""
52     device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
53     prompts = [sample["prompt"] if isinstance(sample, dict) else sample[0] for
sample in batch_samples]
54     answers = [sample["answer"] if isinstance(sample, dict) else sample[1] for
sample in batch_samples]
55
56     with torch.no_grad():
57         prompt_ids, prompt_mask, completion_ids, completion_mask =
generate_completions(
58             model, tokenizer, prompts, num_generations, max_completion_length
59         )
60         input_ids = torch.cat([prompt_ids, completion_ids], dim=1)
61         attention_mask = torch.cat([prompt_mask, completion_mask], dim=1)
62         logits_to_keep = completion_ids.size(1)
63
64         old_log_probs = compute_log_probs(model, input_ids, attention_mask,
logits_to_keep)

```

```

65         ref_log_probs = compute_log_probs(ref_model, input_ids,
        attention_mask, logits_to_keep)
66
67         formatted_completions = [[{'content': tokenizer.decode(ids,
        skip_special_tokens=True)}] for ids in completion_ids]
68         repeated_prompts = [p for p in prompts for _ in range(num_generations)]
69         repeated_answers = [a for a in answers for _ in range(num_generations)]
70
71         return {
72             "input_ids": input_ids,
73             "attention_mask": attention_mask,
74             "completion_mask": completion_mask.

```

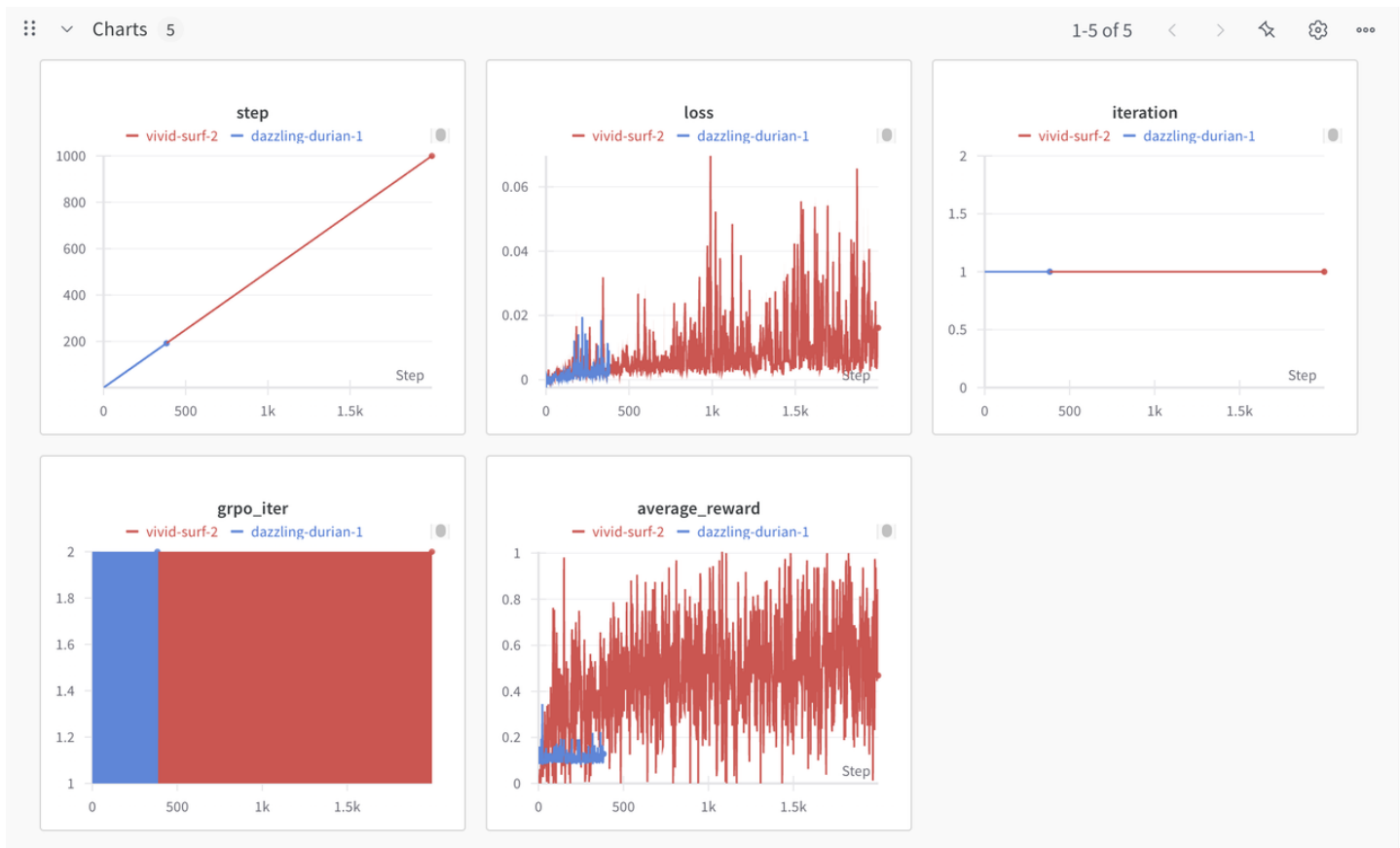
6.3 实验问题与改进策略

在GRPO微调过程中，笔者遇到了一个显著影响训练效果的问题。初期训练时，观察到奖励函数的输出异常偏低，进一步分析发现模型生成的响应缺乏结构化的思考过程表达。通过对训练日志的检查，笔者发现这一现象的根本原因在于系统提示（System Prompt）设计的缺陷——笔者未能在提示中明确要求模型使用指定的XML标签（如 `<reasoning>` 和 `<answer>`）来格式化输出。

这一疏忽直接导致了格式奖励（format_reward）组件几乎无法发挥作用，使得综合奖励信号难以有效引导模型产生结构化的推理过程。从训练曲线上看，这表现为蓝色曲线在早期阶段长时间维持在较低水平，模型优化方向不明确，训练效率大幅降低。

针对这一问题，笔者对系统提示进行了有针对性的重新设计，显式要求模型必须使用特定的XML标签包装其思考过程和最终答案。具体而言，修订后的提示明确指导模型首先在 `<reasoning>` 标签内展开其对前提和假设关系的分析，然后在 `<answer>` 标签中给出最终的分类结果（entailment、contradiction或neutral）。这种结构化输出不仅便于奖励函数的评估，也使模型的推理过程更加透明可解释。

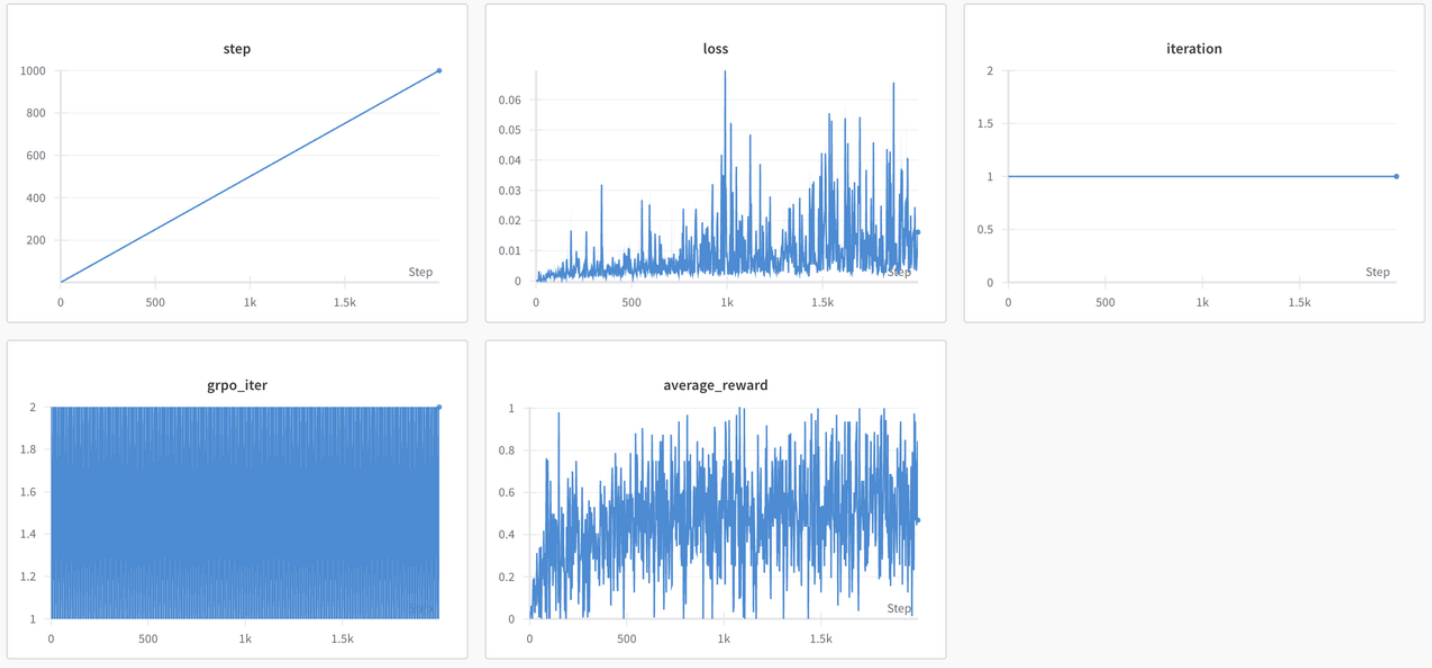
实施这一改进后，训练指标出现了显著变化。如图所示，平均奖励值（average_reward）迅速上升并达到更高的稳定水平，表明模型成功捕捉到了预期的输出格式并开始生成高质量的结构化响应。更值得注意的是，这一简单而有效的干预不仅提高了训练效率，还从根本上改善了模型在验证集上的表现，尤其在中立（neutral）关系判断这一原本较为薄弱的方面取得了显著进步。



两次实验情况对比图



第一次实验情况（图一蓝色）



第二次实验情况（图一红色）

6.4 实验结果与分析

6.5 实验结果：

模型	总正确率 (snli-test)	蕴含	矛盾	中立
Qwen2.5-1.5B-Instruct	44%	96%	13%	0%
Qwen2.5-1.5B-Instruct(SFT-finetune)	67%	98%	15%	0%
Qwen2.5-1.5B-Instruct(GRPO-finetune)	71%	99%	71%	63%

6.5.1 示例分析

6.5.1.1 例1：矛盾的情况

前提:

▼ 代码块

```
1 "This church choir sings to the masses as they sing joyous songs from the
  book at a church."
```

假设:

▼ 代码块

```
1 "A choir singing at a baseball game."
```

正确标签: "contradiction"

1. Qwen2.5-1.5B-Instruct:

▼ 代码块

```
1 contradiction\n\n理由(Rationale): 前提中提到的"two brown dogs"和"two dogs"并不完全一致, 因为前者的数量是两对, 而后者的数量是一对。因此, 这两个句子之间存在矛盾。
```

2. Qwen2.5-1.5B-Instruct(SFT-finetune):

▼ 代码块

```
1 contradiction
```

3. Qwen2.5-1.5B-Instruct(GRPO-finetune):

▼ 代码块

```
1 <thinking>
2 前提是教堂合唱团在教堂里向群众唱欢乐的歌曲。而假设是合唱团在棒球比赛上唱歌。地点上前提中是"教堂"(church), 假设中是"棒球比赛"(baseball game), 这两个地点在物理上不可能同时成立。两个陈述之间存在明确的矛盾, 因为同一个合唱团不可能同时在教堂和棒球场演唱。
3 </thinking>
4 contradiction
```

6.5.1.2 例2: 蕴含的情况

前提:

▼ 代码块

```
1 "A man with long hair getting stopped by security guards."
```

假设:

▼ 代码块

```
1 "The man has long hair."
```

正确标签: entailment

1. Qwen2.5-1.5B-Instruct:

▼ 代码块

```
1 entailment\n\n理由：前提中提到的"man with long hair"直接说明了假设中的"the man has long hair." 因此，前后的陈述是相关的，存在蕴含关系。故答案为: entailment.
```

2. Qwen2.5-1.5B-Instruct(SFT-finetune):

▼ 代码块

```
1 entailment
```

3. Qwen2.5-1.5B-Instruct(GRPO-finetune):

▼ 代码块

```
1 <think>
2 我需要分析这两个句子之间的关系：\n前提(Premise): "A man with long hair getting stopped by security guards." 假设(Hypothesis): "The man has long hair."\n我将逐步分析这两个陈述之间的关系：1. 前提描述了一个留长发的男子被保安拦下的情景。2. 假设单纯陈述这名男子有长发。让我检查这三种可能的关系：蕴含(Entailment): 如果前提为真，那么假设必然为真。矛盾(Contradiction): 如果前提为真，那么假设必然为假。无关(Neutral): 前提的真假不能确定假设的真假。在这个例子中： 前提明确描述了一个"man with long hair"(留长发的男子)，这直接表明这名男子确实拥有长发。因此，假设"The man has long hair"(这名男子有长发)是直接从前提中提取的信息。
\n如果前提为真，那么假设必然为真，因为前提中已经明确说明了这名男子有长发这一特征。
\n因此，这是一个典型的**蕴含(Entailment)**关系。前提中的信息直接蕴含了假设中的陈述。
3 </think>
4 Entailment
```

6.5.1.3 例3：无关的情况

前提:

▼ 代码块

```
1 "A potter making a vase on a spinning wheel."
```

假设:

▼ 代码块
1 `the potter's wheel spins."`

正确标签: neutral

1. Qwen2.5-1.5B-Instruct: ❌

▼ 代码块

- 1 **Contradiction**\n\n理由:\n前提陈述了"一个陶艺师正在使用旋转的轮子制作一个花瓶"。假设说"陶艺师的轮子在转动"。这两个陈述之间存在矛盾,因为前提中提到的是一个"旋转的轮子",而假设只说这个轮子在转动。因此,前提和假设是不一致的,所以答案是contradiction。

2. Qwen2.5-1.5B-Instruct(SFT-finetune): ❌

▼ 代码块

- 1 **entailment**

3. Qwen2.5-1.5B-Instruct(GRPO-finetune): ✔️

▼ 代码块

- 1 `<think>`
- 2 前提中提到"a potter making a vase on a spinning wheel" (陶工在旋转轮上制作花瓶), 这里的"spinning wheel" (旋转轮) 是指这种类型的设备, 是制陶用的工具名称。但假设中并没有指明旋转的轮子是什么, 因此无法判断前提和假设的关系。
- 3 `</think>`
- 4 **neutral**

6.5.2 结果分析

6.5.2.1 模型性能对比分析

从总体性能来看, 三个模型版本呈现出明显的进步趋势:

- 基础模型 Qwen2.5-1.5B-Instruct 的总正确率为44%
- SFT微调后的模型提升至67% (+23%)
- GRPO微调后的模型进一步提升至71% (+4%)

6.5.2.2 各类任务表现对比分析

1. 蕴含关系识别:

所有模型在识别蕴含关系方面表现优异，基础模型已达96%的准确率。而SFT和GRPO微调分别将准确率提升至98%和99%。例2展示了所有模型都能正确识别简单的蕴含关系。

2. 矛盾关系识别：

基础模型在矛盾关系识别上表现较差，仅有13%的准确率，SFT微调后略有提升至15%（+2%），而GRPO微调带来显著突破，准确率大幅提升至71%（+56%）。例1显示虽然在给出的特定示例中三个模型都正确识别了矛盾，但整体统计数据表明基础模型和SFT模型在更复杂的矛盾案例中仍有显著缺陷。

3. 中立关系识别：

基础模型和SFT微调模型完全无法识别中立关系（0%准确率），而GRPO微调后取得重大突破，准确率提升至63%。例3充分展示了这一点：基础模型将中立误判为"矛盾"，SFT模型将中立误判为"蕴含"，只有GRPO模型正确识别为"中立"。

6.5.2.3 各方法提升分析

1. SFT效果：

SFT主要提升了总体准确率（+23%），在蕴含关系识别上有微小提升（+2%），在矛盾关系上提升有限（+2%）对中立关系识别无改善。这表明SFT主要优化了模型在已有强项（蕴含识别）上的表现，但未能有效解决弱项问题。

2. GRPO效果：

GRPO方法全面提升了各类关系的识别能力，特别在模型原本薄弱的矛盾和中立关系识别上取得了质的飞跃，这可能是因为GRPO通过基于规则的优化策略，有效纠正了模型在复杂逻辑关系上的偏差。

6.5.3 总结

实验结果清晰地表明，虽然三个模型版本在蕴含关系识别上都表现良好，但在处理矛盾和中立关系时有显著差异。GRPO微调策略相比传统SFT微调更能有效提升模型在自然语言推理任务上的全面能力，特别是在处理逻辑上更为微妙的中立和矛盾关系时。这一发现对于提升大型语言模型在推理任务上的平衡性能具有重要指导意义。

7. 无需微调，显著提升：多轮对话机制

7.1 实验设计

笔者设计了两组实验来评估对话历史对大模型在海龟汤游戏中表现的影响：

实验1（无对话历史）：在这个实验中，模型仅基于汤面、汤底和当前问题作出判断，不考虑之前的问答历史。这模拟了静态的文本蕴含任务。

实验2（有对话历史）：在这个实验中，模型可以访问之前所有的问答对，模拟真实的多轮对话场景。

所有实验都在三个主流闭源大模型与第五节的两个模型上进行，使用：

- Claude 3.7 (Anthropic)
- GPT-4o (OpenAI)
- ChatGLM4 (智谱AI)
- Qwen2.5-1.5B-Instruct (本地)
- Qwen2.5-1.5B-Instruct(GRPO-finetune) (本地)

为了确保实验的一致性，笔者为所有模型使用相同的系统提示（system prompt），指导模型扮演海龟汤游戏的主持人角色。

笔者的实验流程包括以下步骤：

1. 加载海龟汤数据集
2. 分别在有无对话历史的条件下，让大模型对每个问题进行回答
3. 将模型回答分类为"是"、"不是"或"这个问题与解谜无关"
4. 与标准答案比较，计算评估指标
5. 提取和分析错误案例

以下是实验脚本的核心部分：

7.1.1 实验1（无对话历史）实现

▼ 代码块

```
1  def run_experiment(dataset_path: str, output_path: str, llm_provider:
    LLMProvider):
2      # 确保LLM提供者已初始化
3      llm_provider.initialize()
4
5      # 加载数据集
6      puzzles = load_dataset(dataset_path)
7
8      results = []
9
10     for puzzle in puzzles:
11         puzzle_results = {
12             "title": puzzle["title"],
13             "surface": puzzle["surface"],
14             "bottom": puzzle["bottom"],
15             "questions": []
16         }
17
18         # 创建系统提示
19         system_prompt = create_system_prompt(puzzle["surface"],
            puzzle["bottom"])
```

```

20
21     # 处理思考链中的每个问题
22     for round_idx, round_data in enumerate(puzzle["thought_chain"]
23 ["rounds"]):
24         question = round_data["question"]
25         expected_answer = round_data["answer"]
26
27         # 检查问题格式, 跳过不正确格式的问题
28         if not isinstance(question, str) or question.strip() == "":
29             print(f"跳过空问题或无效问题, 轮次: {round_idx + 1}")
30             continue
31
32         # 分离问题文本和实际问题
33         question_lines = question.strip().split('\n')
34         actual_question = question_lines[-1] if question_lines else
question
35
36         # 使用LLM提供者发送问题
37         llm_response = llm_provider.ask_question(system_prompt,
actual_question)
38         print(f"轮次 {round_idx + 1}: 问题: {actual_question}")
39         print(f"LLM回应: {llm_response}")
40
41         # 分类回答
42         classified_response = classify_response(llm_response)
43
44         # 规范化预期答案以便准确比较
45         normalized_expected = normalize_answer(expected_answer)
46         normalized_classified = normalize_answer(classified_response)
47
48         # 记录结果
49         question_result = {
50             "round": round_idx + 1,
51             "question": question,
52             "expected_answer": expected_answer,
53             "llm_response": llm_response,
54             "classified_response": classified_response,
55             "correct": normalized_classified == normalized_expected
56         }
57
58         print(f"预期答案: '{normalized_expected}', 分类答案:
59 '{normalized_classified}', 正确: {question_result['correct']}")
60         puzzle_results["questions"].append(question_result)
61
62         # 计算此谜题的准确率
63         correct_count = sum(1 for q in puzzle_results["questions"] if
q["correct"])

```

```

62         total_questions = len(puzzle_results["questions"])
63         puzzle_results["accuracy"] = correct_count / total_questions if
total_questions > 0 else 0
64
65         results.append(puzzle_results)
66
67         # 计算总体准确率
68         all_questions = [q for puzzle in results for q in puzzle["questions"]]
69         overall_accuracy = sum(1 for q in all_questions if q["correct"]) /
len(all_questions) if all_questions else 0
70
71         # 保存结果
72         with open(output_path, "w", encoding="utf-8") as f:
73             json.dump({
74                 "puzzles": results,
75                 "overall_accuracy": overall_accuracy
76             }, f, ensure_ascii=False, indent=2)
77         return overall_accuracy

```

在这个实现中，`create_system_prompt` 函数创建了指导模型扮演海龟汤主持人的系统提示，`classify_response` 函数将模型的自然语言回答分类为标准答案格式。每个问题都是独立处理的，没有考虑之前的问答历史。

7.1.2 实验2（有对话历史）实现

▼ 代码块

```

1  def run_experiment(dataset_path: str, output_path: str, llm_provider:
LLMProvider):
2      # 确保LLM提供者已初始化
3      llm_provider.initialize()
4
5      # 加载数据集
6      puzzles = load_dataset(dataset_path)
7
8      results = []
9
10     for puzzle in puzzles:
11         puzzle_results = {
12             "title": puzzle["title"],
13             "surface": puzzle["surface"],
14             "bottom": puzzle["bottom"],
15             "questions": []
16         }
17
18         # 创建系统提示

```

```
19         system_prompt = create_system_prompt(puzzle["surface"],
puzzle["bottom"])
20
21         # 处理思考链中的每个问题
22         for round_idx, round_data in enumerate(puzzle["thought_chain"]
["rounds"]):
23             question = round_data["question"]
24             expected_answer = round_data["answer"]
25
26             # 构建前序对话记录
27             conversation_history = []
28             for prev_idx in range(round_idx):
29                 prev_question = puzzle["thought_chain"]["rounds"][prev_idx]
["question"]
30                 prev_answer = puzzle["thought_chain"]["rounds"][prev_idx]
["answer"]
31
32                 # 确保对话历史中使用的答案格式正确
33                 formatted_answer = format_answer_for_history(prev_answer)
34
35                 # 添加到对话历史
36                 conversation_history.append({"role": "user", "content":
prev_question})
37                 conversation_history.append({"role": "assistant", "content":
formatted_answer})
38
39                 # 使用LLM提供者发送问题，包含对话历史
40                 llm_response = llm_provider.ask_question(system_prompt, question,
conversation_history)
41
42                 print(f"回合 {round_idx + 1}, 问题: {question}")
43                 print(f"原始期望答案: {expected_answer}")
44                 print(f"标准化期望答案:
{normalize_expected_answer(expected_answer)}")
45                 print(f"回答: {llm_response}")
46
47                 # 分类回答
48                 classified_response = classify_response(llm_response)
49
50                 # 标准化期望答案
51                 normalized_expected_answer =
normalize_expected_answer(expected_answer)
52
53                 # 记录结果
54                 question_result = {
55                     "round": round_idx + 1,
56                     "question": question,
```

```

57         "expected_answer": expected_answer,
58         "normalized_expected_answer": normalized_expected_answer,
59         "llm_response": llm_response,
60         "classified_response": classified_response,
61         "correct": classified_response == normalized_expected_answer,
62         "conversation_history": [
63             {"role": m["role"], "content": m["content"]}
64             for m in conversation_history if m["role"] == "user"
65         ] # 仅保存用户问题用于记录
66     }
67
68     puzzle_results["questions"].append(question_result)
69
70     # 计算此谜题的准确率
71     correct_count = sum(1 for q in puzzle_results["questions"] if
72 q["correct"])
73     total_questions = len(puzzle_results["questions"])
74     puzzle_results["accuracy"] = correct_count / total_questions if
75 total_questions > 0 else 0
76
77     results.append(puzzle_results)

```

与实验1相比，实验2的关键区别在于构建并使用了对话历史。对于每个问题，模型不仅接收当前问题，还接收之前所有的问答对，使其能够利用先前交流中的信息。

7.2 结果分析实现

笔者使用以下脚本对实验结果进行分析：

▼ 代码块

```

1  def analyze_experiment_results(input_file: str, output_file: str):
2      """分析实验结果并保存详细报告"""
3      # 加载实验结果
4      results = load_experiment_results(input_file)
5
6      # 将结果转换为DataFrame
7      df = create_analysis_dataframe(results)
8
9      # 重新计算总正确率
10     overall_accuracy = df["correct"].mean()
11
12     # 计算F1分数
13     f1_scores = calculate_f1_scores(df)
14
15     # 创建混淆矩阵

```

```
16 cm, classes = create_confusion_matrix(df)
17
18 # 计算每轮的正确率
19 round_accuracy = calculate_round_accuracy(df)
20
21 # 计算每个谜题的F1分数
22 puzzle_f1_scores = {}
23 for puzzle_title in df["puzzle_title"].unique():
24     puzzle_df = df[df["puzzle_title"] == puzzle_title]
25     puzzle_f1_scores[puzzle_title] = calculate_f1_scores(puzzle_df)
26
27 # 提取错误案例
28 error_cases = extract_error_cases(results)
29
30 # 创建分析报告
31 analysis_report = {
32     "overall_accuracy": overall_accuracy,
33     "f1_scores": f1_scores,
34     "confusion_matrix": cm.tolist(),
35     "confusion_matrix_classes": classes,
36     "round_accuracy": round_accuracy,
37     "puzzle_f1_scores": puzzle_f1_scores,
38     "error_cases": error_cases
39 }
40
41 # 保存分析报告
42 with open(output_file, "w", encoding="utf-8") as f:
43     json.dump(analysis_report, f, ensure_ascii=False, indent=2)
44 return analysis_report
```

分析脚本计算了多种评估指标，包括准确率、F1值、混淆矩阵和轮次准确率，并提取了错误案例供进一步分析。

7.3 实验结果与分析

笔者通过对比实验全面评估了对话历史对大语言模型文本蕴含能力的影响。实验数据呈现了颇具启发性的模式与趋势，揭示了模型能力的多个维度。

7.3.1 整体准确率

表1和图1显示了三个模型在两组实验中的整体准确率。

模型	实验1（无对话历史）	实验2（有对话历史）	提升

Claude 3.7	92.83%	96.62%	0.0379
GPT-4o	94.94%	94.09%	-0.85
ChatGLM4	84.81%	92.83%	0.0802
Qwen2.5-1.5B	50.63%	71.31%	0.2068
Qwen2.5-1.5B(GRPO)	52.32%	71.31%	0.1899

表1: 不同模型在有无对话历史条件下的准确率比较

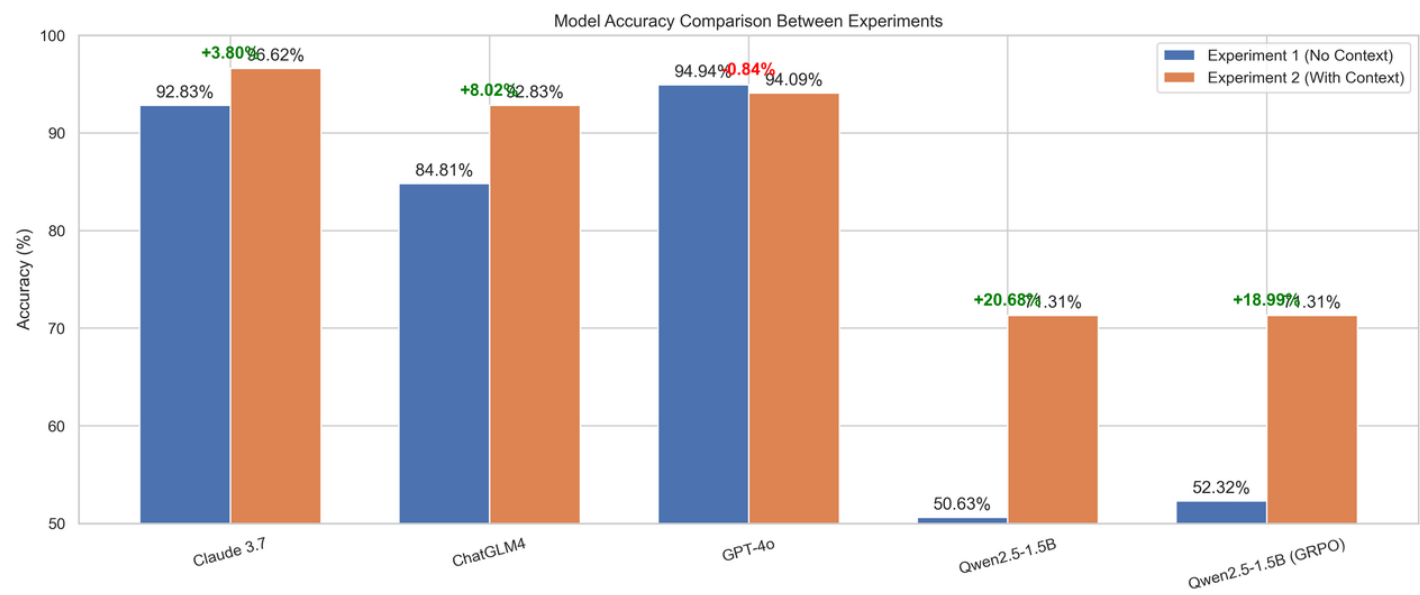


图1: 不同模型在有无对话历史条件下的准确率比较

从整体准确率来看，在无对话历史条件下，GPT-4o以94.94%的准确率领先，其次是Claude 3.7（92.83%）和ChatGLM4（84.81%），而本地部署的Qwen2.5-1.5B系列模型则表现相对薄弱。引入对话历史后，各模型表现出明显的分化趋势：Claude 3.7的准确率提升至96.62%，ChatGLM4的性能跃升8.02个百分点，展现出对历史信息的高效利用能力；而值得关注的是，GPT-4o的表现略有下降，这一反直觉的现象可能暗示其在处理长上下文时的优化空间。

7.3.2 F1值比较

表2展示了三个模型在两组实验中各类别回答的F1值。

模型	实验	"是" F1	"不是" F1	"无关" F1	宏平均F1
Claude 3.7	实验1	0.96	0.6667	0	0.5422
Claude 3.7	实验2	0.9788	0.8085	1	0.9291
GPT-4o	实验1	0.974	0.8085	0.5	0.7608

GPT-4o	实验2	0.9693	0.766	0	0.5784
ChatGLM4	实验1	0.9668	0.72	1	0.8956
ChatGLM4	实验2	0.9619	0.72	0	0.5606

表2: 不同模型在有无对话历史条件下的F1值比较

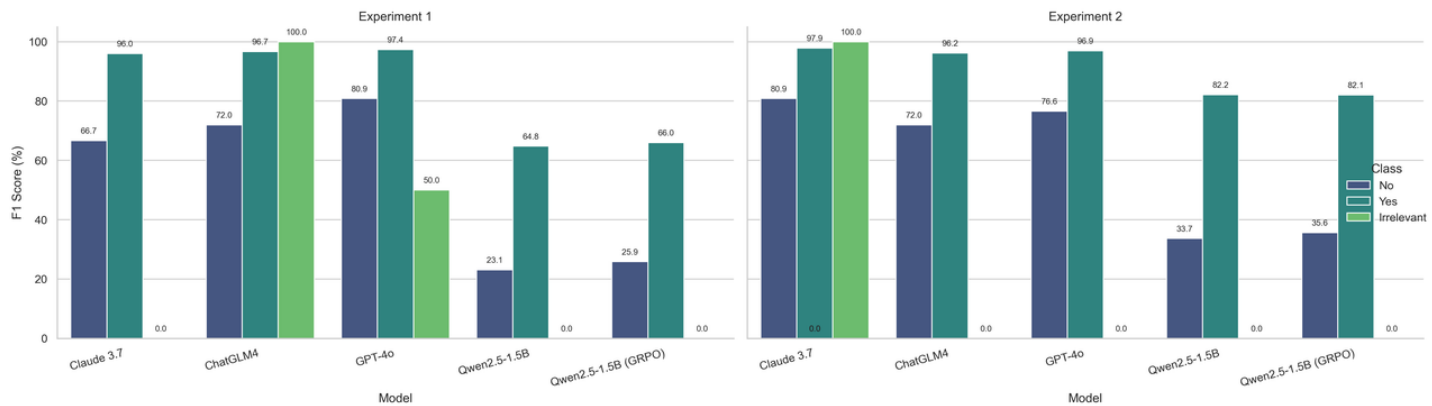


图2: 不同模型在有无对话历史条件下的F1值比较

F1值分析进一步细化了模型在不同类别判断上的能力差异。所有模型在"是"类判断上表现优异，F1值普遍超过0.96；而"不是"类别则构成了更严峻的挑战，尤其在无对话历史时表现不佳；"无关"类别数量稀少，导致评估结果波动较大。特别引人注目的是Claude 3.7在有对话历史条件下，"无关"类别的F1值从0跃升至1.0，展示了多轮对话对边缘情况判断的显著增益。轮次准确率的演变轨迹揭示了模型能力的动态变化：早期轮次（1-3轮）普遍表现不佳，反映了信息不足时推理的困难性；随着对话深入，模型性能呈现上升趋势，这种"学习曲线"与人类认知过程颇为相似；有对话历史的实验组在中后期轮次（4-7轮）表现明显优于无历史组，证实了累积信息对推理质量的提升作用。

7.3.3 轮次准确率

图1展示了模型在各轮对话中的准确率变化。

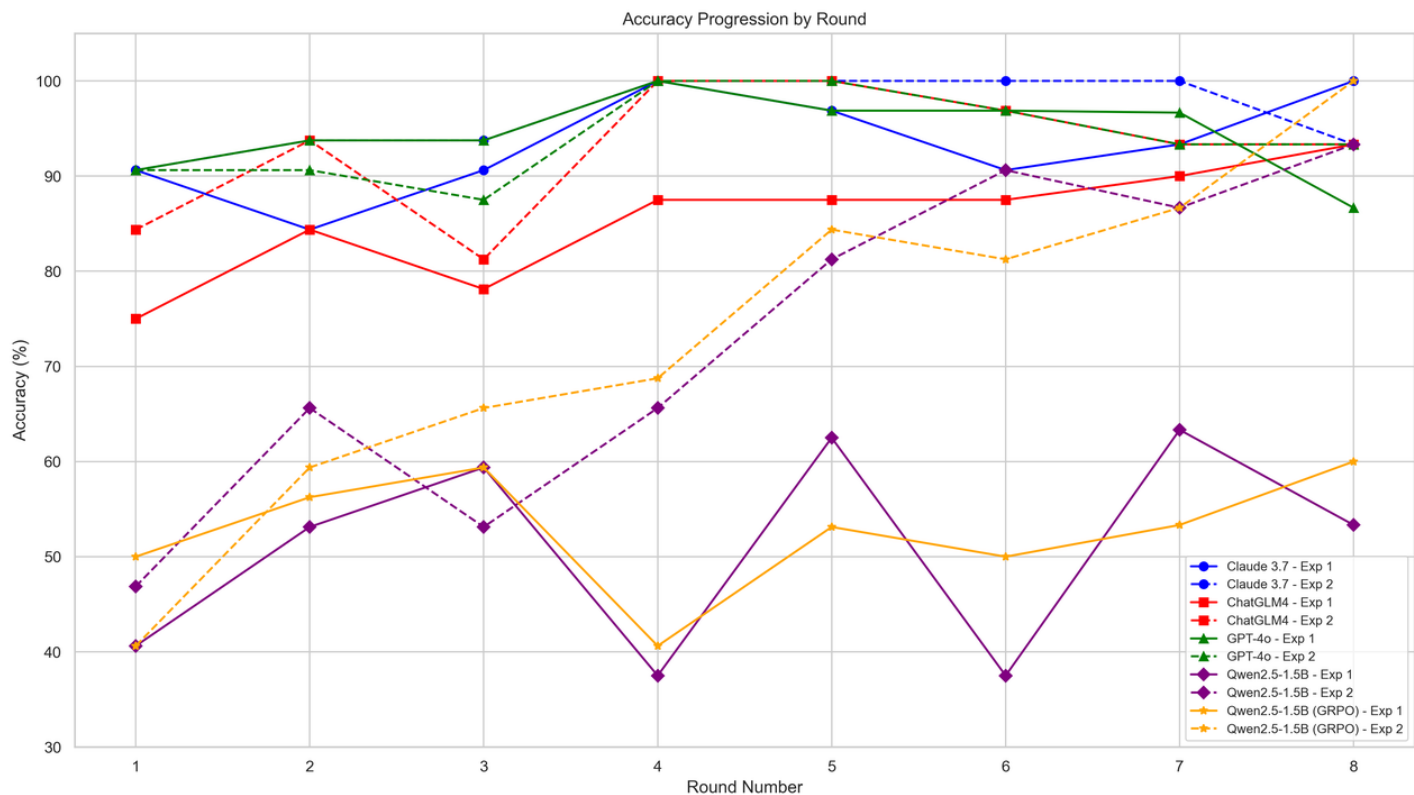


图1: 各模型在不同轮次的准确率变化

从轮次准确率图可以看出：

1. 在早期轮次（尤其是第1-3轮），模型准确率相对较低，这可能是因为上下文信息较少，推理难度较大。
2. 随着对话的进行，所有模型的准确率总体呈上升趋势，这表明模型能够从对话中积累信息。
3. 在实验2（有对话历史）中，模型在中后期轮次（第4-7轮）的准确率普遍高于实验1，特别是 Claude 3.7和ChatGLM4。
4. 在第8轮（最后一轮），准确率出现了一定程度的下降，这可能是因为最后一轮问题通常是对整个谜底的总结性验证，模型需要综合判断用户的回答是否构成完整的汤底，复杂度较高。

7.3.4 谜题难度分析

图2展示了不同谜题的难度，以模型在该谜题上的平均F1值衡量。

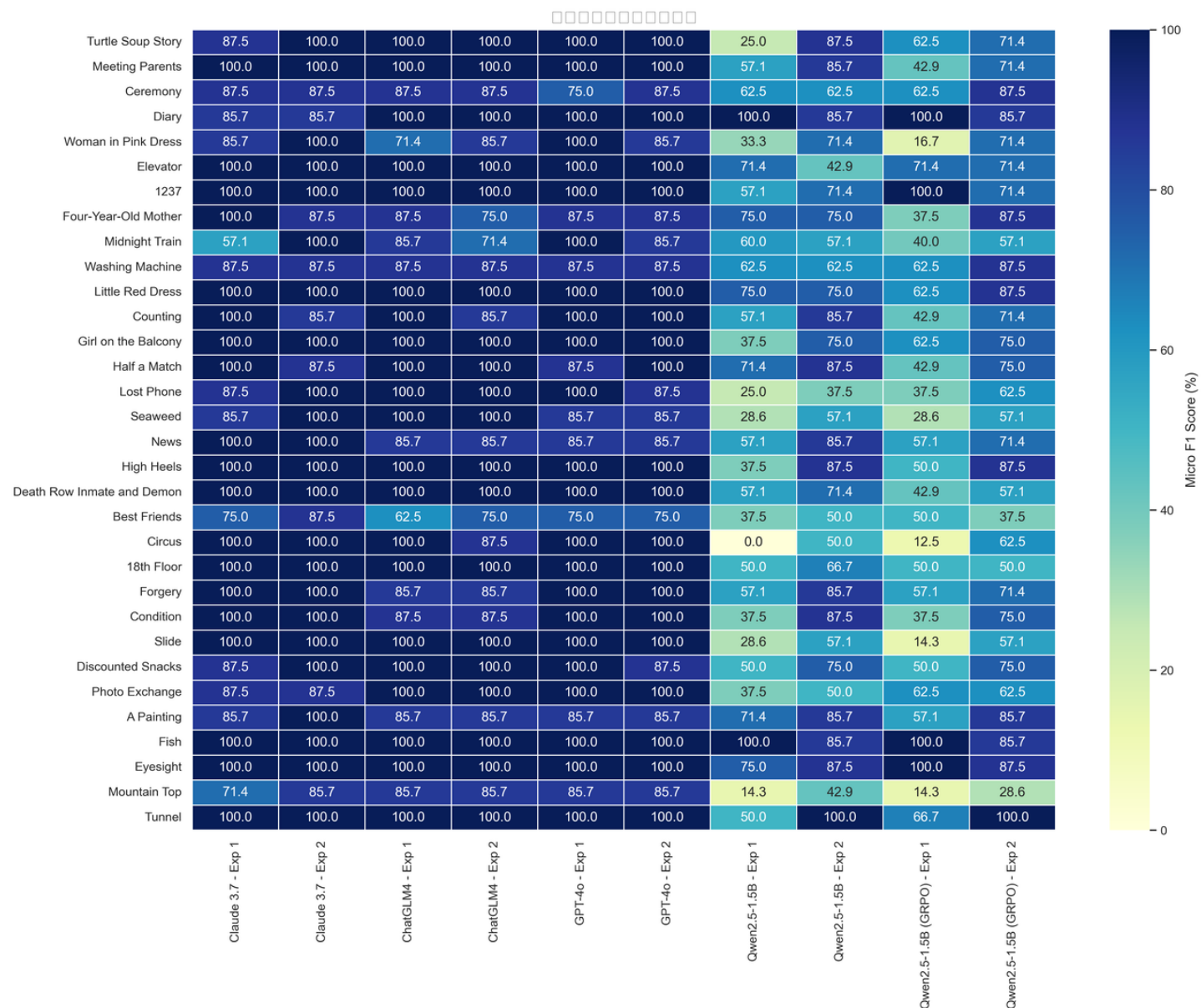
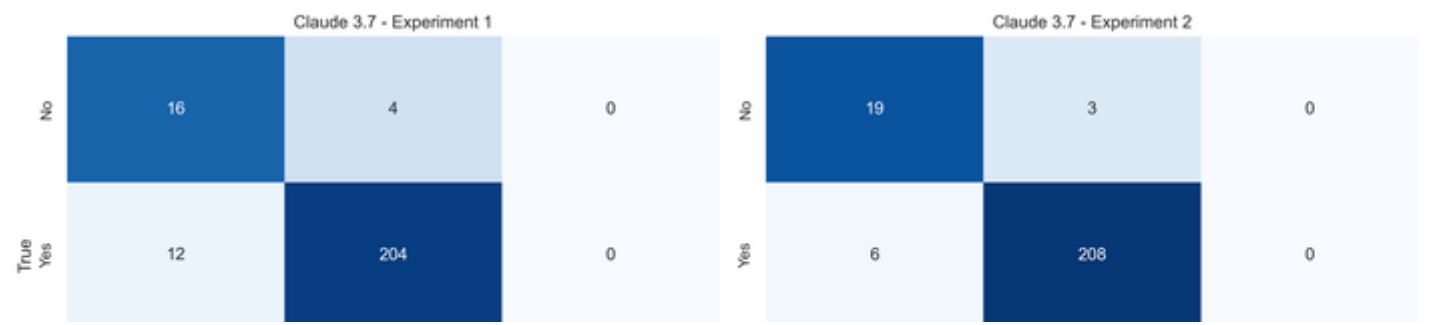


图2: 不同谜题的难度比较

不同谜题的难度分析生动展示了模型面对各类推理挑战的适应性。"午夜列车"、"山顶"、"闺蜜"和"四岁的妈妈"等谜题构成了最具挑战性的测试场景，要求模型在社会关系、空间位置和抽象概念等维度进行复杂推理；而"电梯"、"半根火柴"等相对直接的场景则获得了较高的解决率。混淆矩阵分析揭示了模型的系统性偏误：所有模型普遍倾向于将"不是"错误分类为"是"，这种偏向性在引入对话历史后有所缓解但并未完全消除。

7.3.5 混淆矩阵分析

图3展示了各模型在两组实验中的混淆矩阵。



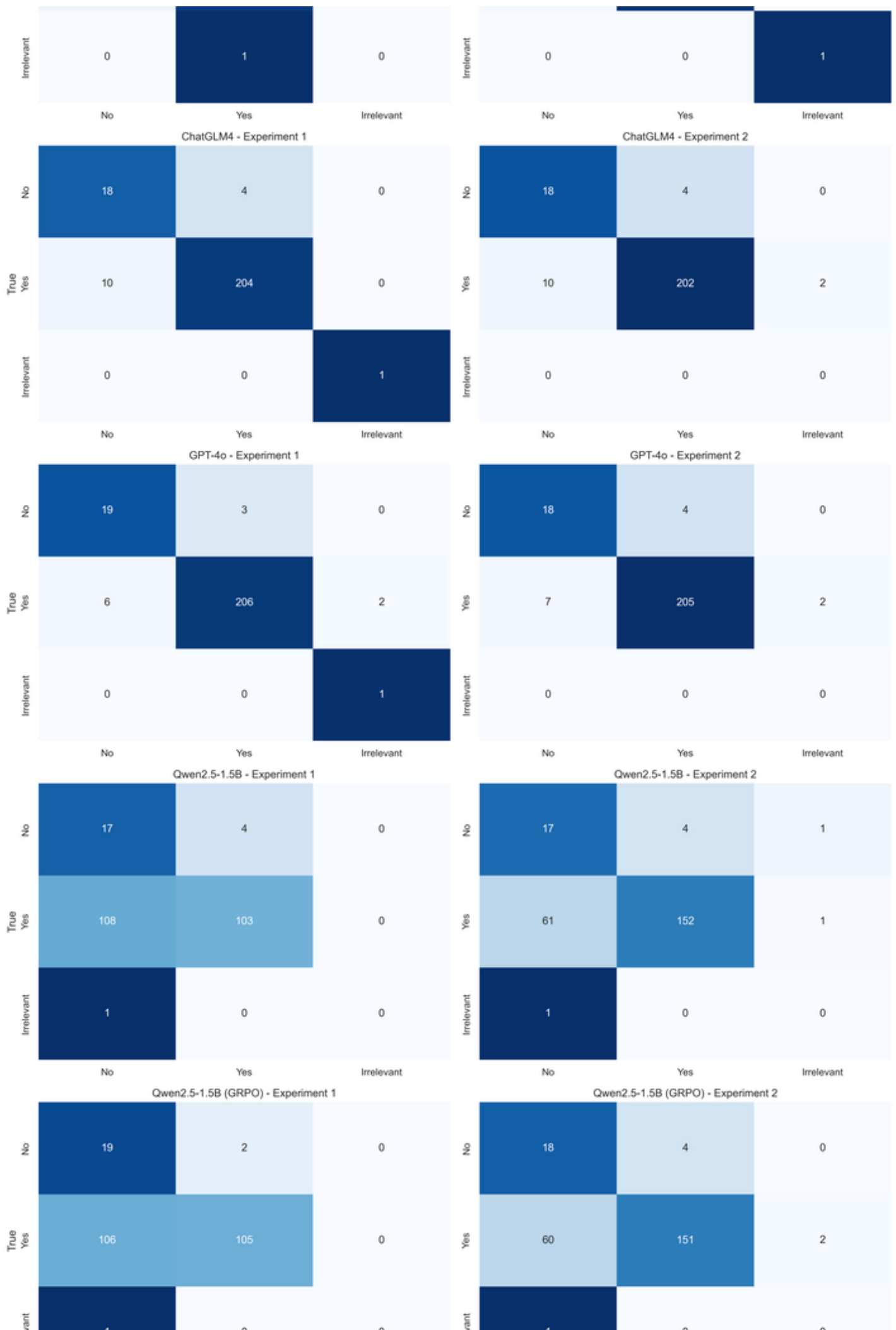




图3: 混淆矩阵比较

从混淆矩阵可以观察到：

- 1. 所有模型在判断"是"类问题时表现良好，错误率较低。
- 2. 模型普遍倾向于将"不是"错误分类为"是"，而较少将"是"错误分类为"不是"。
- 3. "无关"类别的问题数量很少，模型对此类问题的处理能力难以准确评估。
- 4. 在有对话历史的实验中，所有模型在处理"不是"类别问题时的准确率都有提升，误分类为"是"的情况减少。

7.3.6 错误样例分析

通过分析实验中的错误案例，笔者发现了一些模式化的错误类型，这些错误揭示了大模型在文本蕴含任务中面临的挑战。

7.3.6.1 社会关系推理错误

在"闺蜜"谜题中，所有模型都在判断关于社会关系的问题时出现了错误。例如：

示例1: 社会关系推理错误

▼ 代码块

```
1  谜题：闺蜜
2  汤面：托马斯第一次和老婆去老婆闺蜜家，结果回来老婆就要和他离婚了，请问为什么？
3  汤底：托马斯老婆看见老公的手机自动连上了闺蜜家的WiFi。
4  问题：托马斯在闺蜜家的行为是导致老婆要离婚的原因吗？
5  预期答案：不是
6  模型回答：是
```

在这个例子中，模型错误地推断托马斯的行为是导致离婚的原因，而实际上是手机自动连上WiFi这一现象表明托马斯之前来过闺蜜家。这种错误表明模型在理解隐含的社会关系和情感反应时存在困难。

7.3.6.2 空间关系推理错误

在"山顶"谜题中，模型在处理空间关系时出现了错误：

示例3: 空间关系推理错误

▼ 代码块

```
1  谜题：山顶
```

- 2 汤面：一个人住在山顶的小屋里，半夜听见有敲门声音，但是他打开门却没有人，于是去睡了。第二天，有人在山脚下发现死尸一具，请问发生了什么？
- 3 汤底：山顶的小屋的门是悬崖，悬崖下的人好不容易才爬上来，想要敲门求救。一开门，就又被推了下去，最后从山顶上掉下去摔死了。
- 4 问题：住在山顶小屋的人开门时，敲门者是否已经从山顶坠落？
- 5 预期答案：不是
- 6 模型回答：是

这个错误表明模型在理解动作时序和空间位置关系时存在困难，无法正确推断敲门者在门被打开时的位置。

7.3.6.3 模糊概念理解推断错误

在"仪式"谜题中，模型在理解抽象概念时出错：

示例4: 因果关系推断错误

▼ 代码块

- 1 谜题：仪式
- 2 汤面：最近你的父母总是吵架，你也觉得你的母亲状态有些奇怪，她总是心不在焉，偷偷问你怎么给别人转账.....有时候会在家里无声地挥舞双手像在进行什么仪式。一天晚饭过后，你父亲出门去散步了，你母亲也悄悄出了门，已经起了疑心的你跟在她身后.....啊，原来如此.....
- 3 汤底：你母亲跳广场舞，你父亲以为她外面有人了，转账是为了买跳舞用的团队服装，手舞足蹈是为了练习舞蹈。而且为了防止扰民，所有的广场舞阿姨都是戴着耳机无声跳舞的，看起来像是在进行什么神秘仪式似的。
- 4 问题：母亲的异常行为是否与某种经济问题有关？
- 5 预期答案：是
- 6 模型回答：不是

模型错误地认为母亲的行为与经济问题无关，而实际上转账是为了购买舞蹈服装，这实际上属于一种经济问题，因此理解上存在问题。

7.3.7 总结

总体而言，实验结果强有力地支持了对话历史对大模型文本蕴含能力的积极影响。这种影响表现为四个维度：语境累积效应使模型能在交互过程中构建更丰富的理解基础；不同模型架构对历史信息的利用能力存在显著差异；对话中的正确信息能有效防止后续轮次中出现类似错误；海龟汤这种渐进式推理任务与多轮对话机制的高度契合，进一步放大了历史信息的价值。